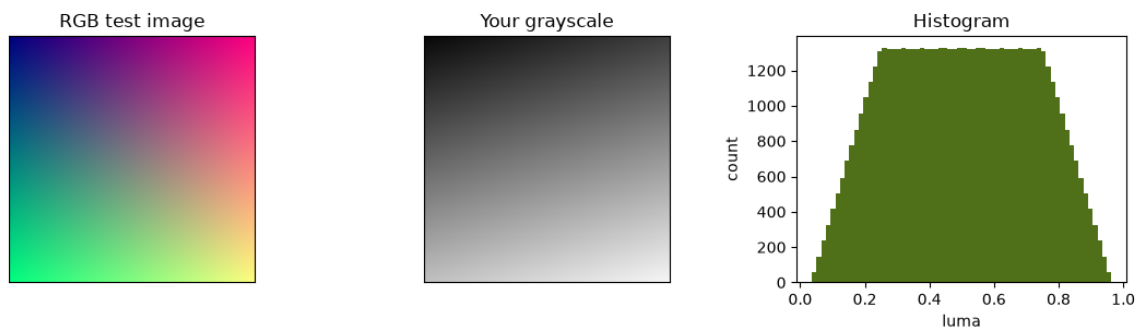


Marco Martinez Corral marcom@pdx.edu EE513 Assignment 1, Setup and First Captures.

Assignment Tasks

1: Build the course environment from the two files on the setup page and run the smoke test notebook. Deliverable: the version block, the smoke test prints, showing Python, NumPy, and OpenCV versions. **Deliverables:**

```
Python    3.12.14
Platform  macOS-27.0.1-arm64-arm-64bit
Executable /Users/marco/Documents/PSU/EE513/.venv/bin/python
Python version: 3.12.14 (main, Aug 12 2026, 13:57:54) [Clang 21.0.0
(clang-2100.1.1.101)]
Python executable: /Users/marco/Documents/PSU/EE513/.venv/bin/python
[PASS] all required packages import numpy 2.5.3, scipy 1.18.1,
matplotlib 3.11.2, cv2 5.0.0, rawpy 0.27.1, imageio 2.38.0, skimage
0.26.0
[PASS] OpenCV major version is 5 found 5.0.0
[PASS] functions this course uses are present 7 of 7
```



2: GitHub Repository URL: <https://github.com/MarcoMartinezC/EE513Assignment1>

3: Capture one raw photograph of anything, load it with rawpy, and print `raw_type`, the shape and dtype of `raw_image_visible`, `raw_pattern`, `color_desc`, `black_level_per_channel`, and `white_level`. Deliverable: the printed output.

Deliverables

```
In [1]: import sys, platform, textwrap

import rawpy
import numpy as np

# RAW image loading using rawpy.imread
# I'm using a NEF file from a Nikon Z50 Camera. Picture is from the Big Ben
raw = rawpy.imread('DSC_6997.NEF')

# Print "raw" variable parameters
print("Raw Type:", raw.raw_type)
print("Shape of raw_image_visible:", raw.raw_image_visible.shape)
```

```
print("Dtype of raw_image_visible:", raw.raw_image_visible.dtype)
print("Raw Pattern:\n", raw.raw_pattern)
print("Color Desc:", raw.color_desc)
print("Black Level per Channel:", raw.black_level_per_channel)
print("White Level:", raw.white_level)
```

```
Raw Type: RawType.Flat
Shape of raw_image_visible: (3728, 5600)
Dtype of raw_image_visible: uint16
Raw Pattern:
[[0 1]
 [3 2]]
Color Desc: b'RGBG'
Black Level per Channel: [1008, 1008, 1008, 1008]
White Level: 16383
```

4. Given that RawType.Flat, a dimension of (3728, 5600) (which matches the DX resolution of my Nikon Z50 camera), and a 2x2 matrix for the Raw pattern, this image has a Bayer mosaic. CFA Layout: b'RGBG'

5. Not applicable

6. *Capture the same scene twice without moving the camera, once as raw and once as an ordinary JPEG or HEIC. Report the file size, bit depth, and number of channels of each.*

When I took the Big Ben picture, my camera was set to save both NEF and JPG file versions of the pictures. Using exiftool on macOS, the NEF image has 14 bit sampling:

```
exiftool -BitsPerSample -NEFCompression DSC_6997.NEF
Bits Per Sample      : 14
NEF Compression      : Lossless
```

Format	File Size	Bit Depth	Channels
.NEF	27.1 MB	14 bit	1
.JPG	5.4 MB	8 bit	3

7. *Display a region of about 30 by 30 pixels of the raw data, zoomed so that individual pixels are visible as blocks. Deliverable: the figure, and a sentence identifying what the checkerboard texture is. It is not noise.*

```
In [2]: import matplotlib.pyplot as plt
import matplotlib.image as mpimg

# Load NEF image and extract 30x30 region
with rawpy.imread('DSC_6997.NEF') as raw:
    raw_crop = raw.raw_image_visible[1000:1030, 1000:1030]

# JPEG loading
jpg_img = mpimg.imread('DSC_6997.JPG')
jpg_crop = jpg_img[1000:1030, 1000:1030]
```

```
# Plots
fig, axes = plt.subplots(1, 2, figsize=(10, 5))

# Plot RAW
axes[0].imshow(raw_crop, cmap='gray', interpolation='nearest')
axes[0].set_title("Blue Sky Big Ben 30x30 RAW (1-Channel Mosaic)")
axes[0].axis('off')

# JPG plot for difference checking, I wanted to see the demosaicing results
# Plot JPG and displays the demosaiced, color-interpolated result
axes[1].imshow(jpg_crop, interpolation='nearest')
axes[1].set_title("Blue Sky Big Ben 30x30 JPEG (3-Channel RGB)")
axes[1].axis('off')

plt.tight_layout()
plt.show()

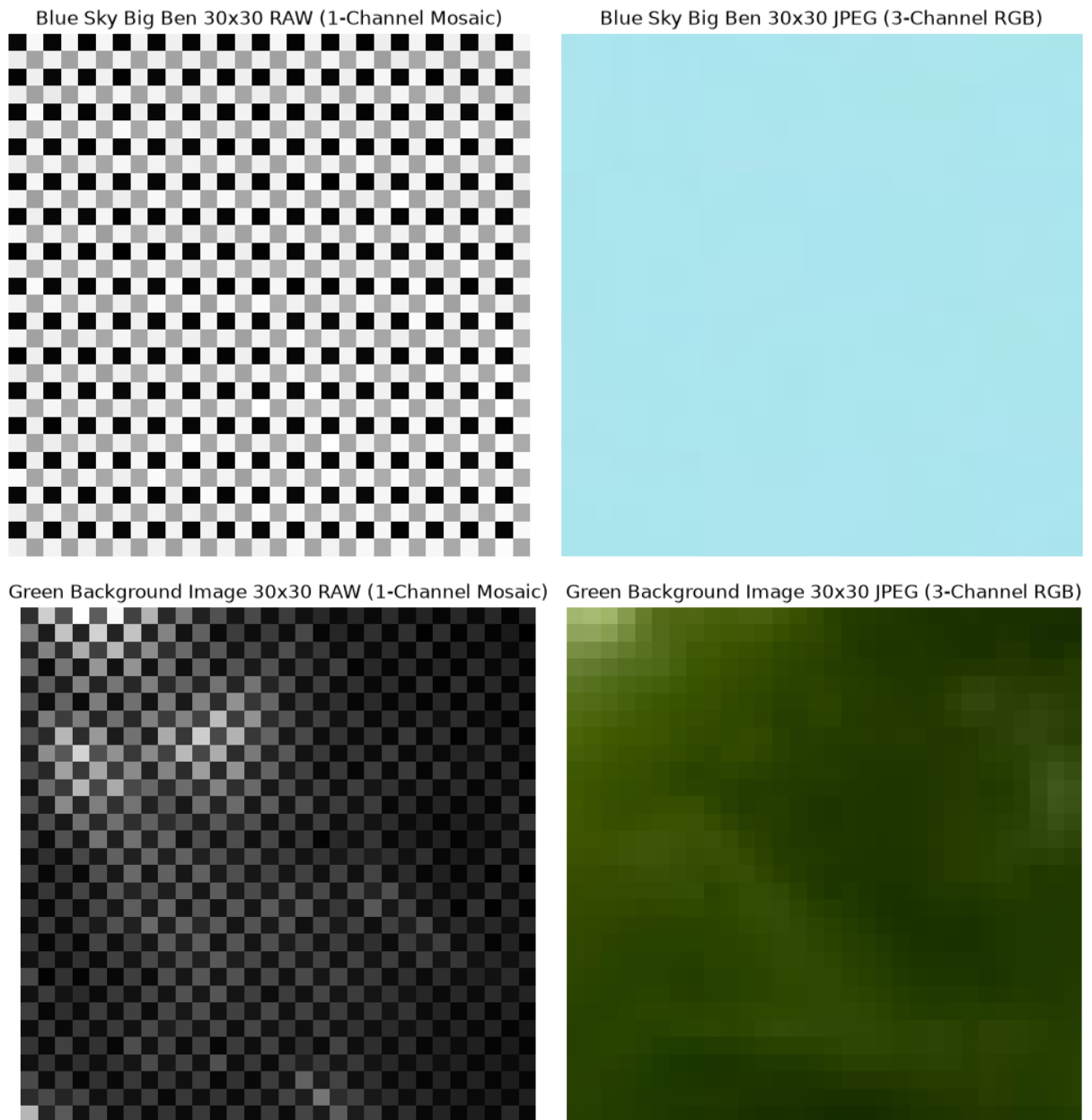
# Testing with additional "green" image since checkerboard for Big Ben image
with rawpy.imread('DSC_7143.NEF') as raw:
    raw_crop = raw.raw_image_visible[1000:1030, 1000:1030]

# JPEG loading
jpg_img = mpimg.imread('DSC_7143.JPG')
jpg_crop = jpg_img[1000:1030, 1000:1030]
fig, axes = plt.subplots(1, 2, figsize=(10, 5))

# Plot RAW
axes[0].imshow(raw_crop, cmap='gray', interpolation='nearest')
axes[0].set_title("Green Background Image 30x30 RAW (1-Channel Mosaic)")
axes[0].axis('off')

# Plot JPEG
axes[1].imshow(jpg_crop, interpolation='nearest')
axes[1].set_title("Green Background Image 30x30 JPEG (3-Channel RGB)")
axes[1].axis('off')

plt.tight_layout()
plt.show()
```



The checkerboard texture in the RAW plot is the physical Color Filter Array (CFA) present on the sensor. Each pixel recorded one color, in this case, the blue profile of the sky background of the Big Ben appears almost uniformly lit for the blue pixels (white squares in the mosaic, in this case), with the green background image of the Union Jack showing a more varied brightness range across the green channel, with lower intensity values than that of the Big Ben image (images shown below).

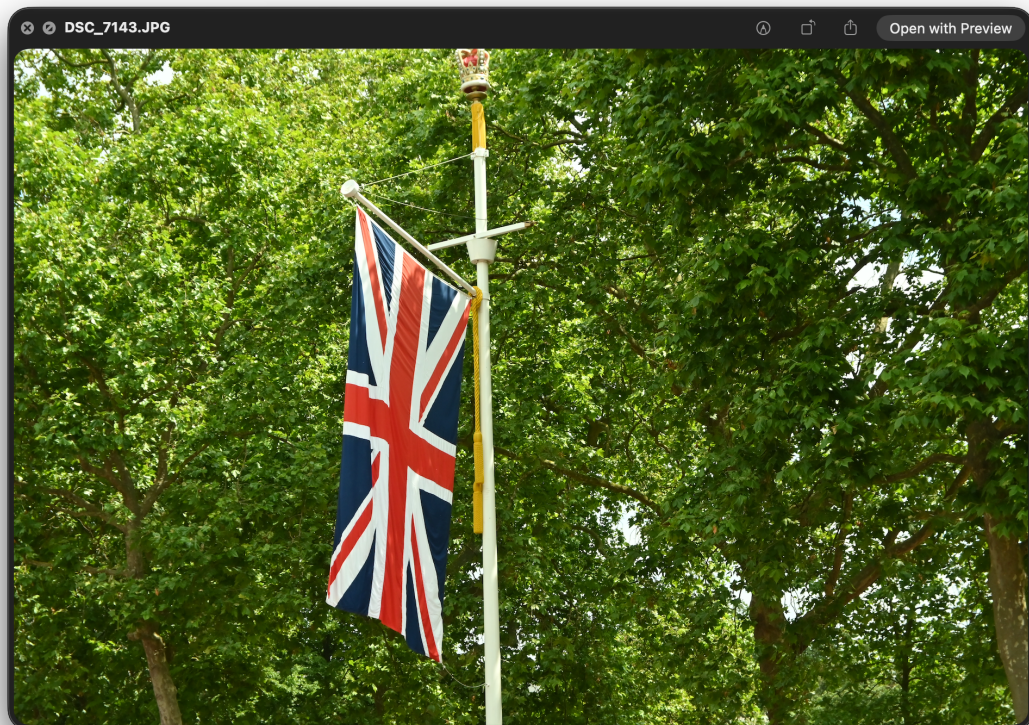
The 1 channel mosaic of the green background does vary in intensity ranges as the green values are brighter, as can be seen near the top left region.

The RGB profile shows the demosaicing applied to the image; this interpolates all 3 colors in the RGB color space, producing light blue and green images for the light blue sky background and green foliage backgrounds, respectively.

JPG preview of the blue sky Big Ben image:



JPG preview of the Green Background Flag image:



9. Print the calibration target at 100% scale and tape it flat to card or board. Measure one square with a ruler and report the measured size in millimetres

Deliverable: the measured square size, and one sentence on why you will use your measured value rather than the nominal 20.0 mm.

Measured square size: 19.9mm. This measured value will be used, as it is the real world/ actual value printed out, as opposed to the pixel value fed into the printer. Since the picture will be of the paper, the actual dimensions will need to be accounted for.

10. Capture at least 15 photographs of the target at a variety of angles, distances, and positions in the frame, including several where the board is tilted well away from parallel and several where it sits near a corner of the frame

11. Run `cv2.findChessboardCorners` with the pattern size (9, 6) on every capture and report how many succeeded. *Deliverable: the count, and one image with the detected corners drawn on it.*

12. Pass the successful detections to `cv2.calibrateCamera` and report the RMS reprojection error it returns. A rough number is fine here; Project 1 is where you will interpret it. *Deliverable: the RMS reprojection error, and one sentence on what it means and what units it is in.*

(See code snippet below for deliverables 10, 11, and 12)

```
In [31]: import sys
        from pathlib import Path

        import cv2
        import matplotlib.pyplot as plt
        import numpy as np

        # As I am not as familiar with Python as I am with SystemVerilog/C (due to t
        # I used Gemini AI to walk me through the methods and functions present in t
        # "02_calibrate_your_camera" notebooks. From there, I tried to use "blocks"

        # 1 set the parameters for the board
        # 2 import files
        # 3 set the contact sheet
        # 4 find corners via a "small image" and applies this to the larger image (t
        # 5 draws the chessboard corners and imposes it on the first successful imag

        # Configuration
        CAPTURE_DIR = "converted_images" # directory with converted JPEG images. I
        SQUARE_MM = 19.9                 # Actual measured physical square size
        BOARD = (9, 6)                   # INNER corners (squares - 1)
        DETECT_SCALE = 0.5                # detect on a smaller copy for speed; ref
        IMG_SUFFIXES = {".jpg", ".jpeg"} # using converted JPG images via macOS op
```

```

# Setting directory for images
folder = Path(CAPTURE_DIR)
if not folder.is_dir():
    sys.exit(f"Folder not found: {folder.resolve()} (run from the right dir)
files = sorted(p for p in folder.iterdir() if p.suffix.lower() in IMG_SUFFIX)
if not files:
    sys.exit(f"No usable images in {folder.resolve()}")
print(f"{len(files)} image files found")

# Error throwing, suggested by Gemini AI
def load_bgr(path):
    """Read an image, or return None (and say why) instead of crashing later
    img = cv2.imread(str(path))
    if img is None:
        print(f" could not decode, skipping: {path.name}")
    return img

# Contact sheet
# Using 24 images for a 3x8 grid so it fits in the window. I deliverately to
# DSC_1206, DSC_1207, DSC_1208, and DSC_1209.
# Not relevant, but the white balance was left "as is", resulting in the ove
n = min(24, len(files))
fig, axes = plt.subplots(3, 8, figsize=(24, 9))
for ax in axes.ravel():
    ax.axis("off")
for ax, f in zip(axes.ravel(), files[:n]):
    img = load_bgr(f)
    if img is None:
        continue
    # this is the region that Claude AI identified as having a potential mem
    # I had loaded where being stored in RAM. The program kept stalling inde
    # was applied.
    thumb = cv2.resize(img, None, fx=0.1, fy=0.1, interpolation=cv2.INTER_AR
    ax.imshow(cv2.cvtColor(thumb, cv2.COLOR_BGR2RGB))
    ax.set_title(f.name, fontsize=7)
fig.suptitle("Contact Sheet of Calibration Captures")
plt.tight_layout()
plt.show()

# Find Corners

# Z coordinate is left as 0. The values from the remaining X and Y coordinat
## the measured physical value of the square, and a grid is built from the a

objp = np.zeros((BOARD[0] * BOARD[1], 3), np.float32)
objp[:, :2] = np.mgrid[0:BOARD[0], 0:BOARD[1]].T.reshape(-1, 2) * SQUARE_MM

obj_points, img_points, used = [], [], []
drawn_img, img_shape = None, None
# "stop conditions" once 30 iterations have occurred or 0.001 pixel shift
crit = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 30, 0.001)
# adaptive thresh and normalize image so that "uneven" captures like the one
flags = cv2.CALIB_CB_ADAPTIVE_THRESH + cv2.CALIB_CB_NORMALIZE_IMAGE

# File import

```

```

for f in files:
    img = load_bgr(f)
    if img is None:
        continue
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    if img_shape is None:
        img_shape = gray.shape[::-1] # (width, height)
    elif gray.shape[::-1] != img_shape: # e.g. portrait v
        print(f" size {gray.shape[::-1]} != {img_shape}, skipping: {f.name}")
        continue
    # resizing array/image so that findChessboardCorners does not use up as man
    small = cv2.resize(gray, None, fx=DETECT_SCALE, fy=DETECT_SCALE, interpo
    ok, corners = cv2.findChessboardCorners(small, BOARD, flags)
    if not ok:
        print(f" no corners: {f.name}")
        continue
    # As I understand it, this is similar to "scale images" where the smalle
    # to larger coordinates
    corners = (corners / DETECT_SCALE).astype(np.float32) # back to full-re
    corners = cv2.cornerSubPix(gray, corners, (11, 11), (-1, -1), crit)
    obj_points.append(objp)
    img_points.append(corners)
    used.append(f)
    if drawn_img is None:
        drawn_img = img.copy()
        cv2.drawChessboardCorners(drawn_img, BOARD, corners, ok)

print(f"Corners found: {len(img_points)} out of {len(files)} images succee

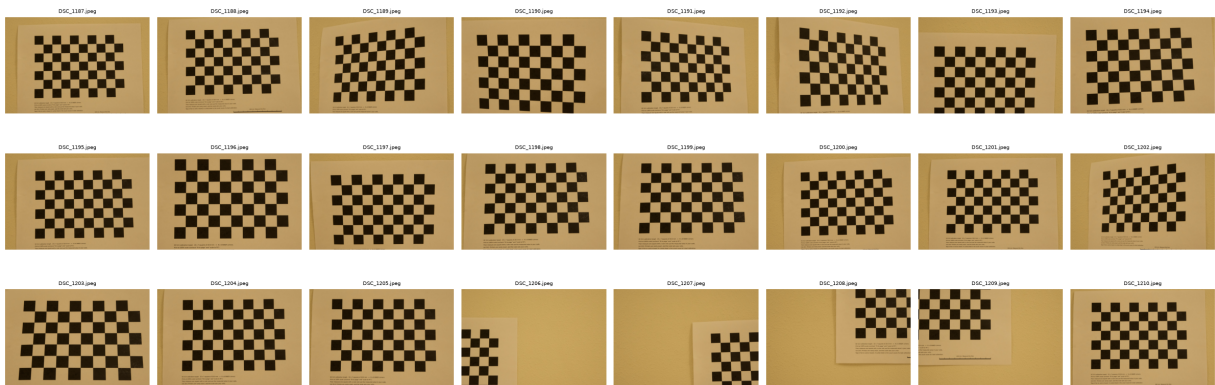
if drawn_img is not None:
    plt.figure(figsize=(10, 7))
    plt.imshow(cv2.cvtColor(drawn_img, cv2.COLOR_BGR2RGB))
    plt.title("Detected Calibration Corners")
    plt.axis("off")
    plt.show()

# RMS Projection Error
if len(img_points) >= 3: # need several, differently tilted views, used
    rms, K, dist, rvecs, tvecs = cv2.calibrateCamera(obj_points, img_points,
    print(f"RMS Reprojection Error = {rms:.4f} px")
    print("K =\n", K, "\ndist =", dist.ravel())
else:
    print("Not enough successful corner detections to calibrate.")

```

24 image files found

Contact Sheet of Calibration Captures



no corners: DSC_1206.jpeg

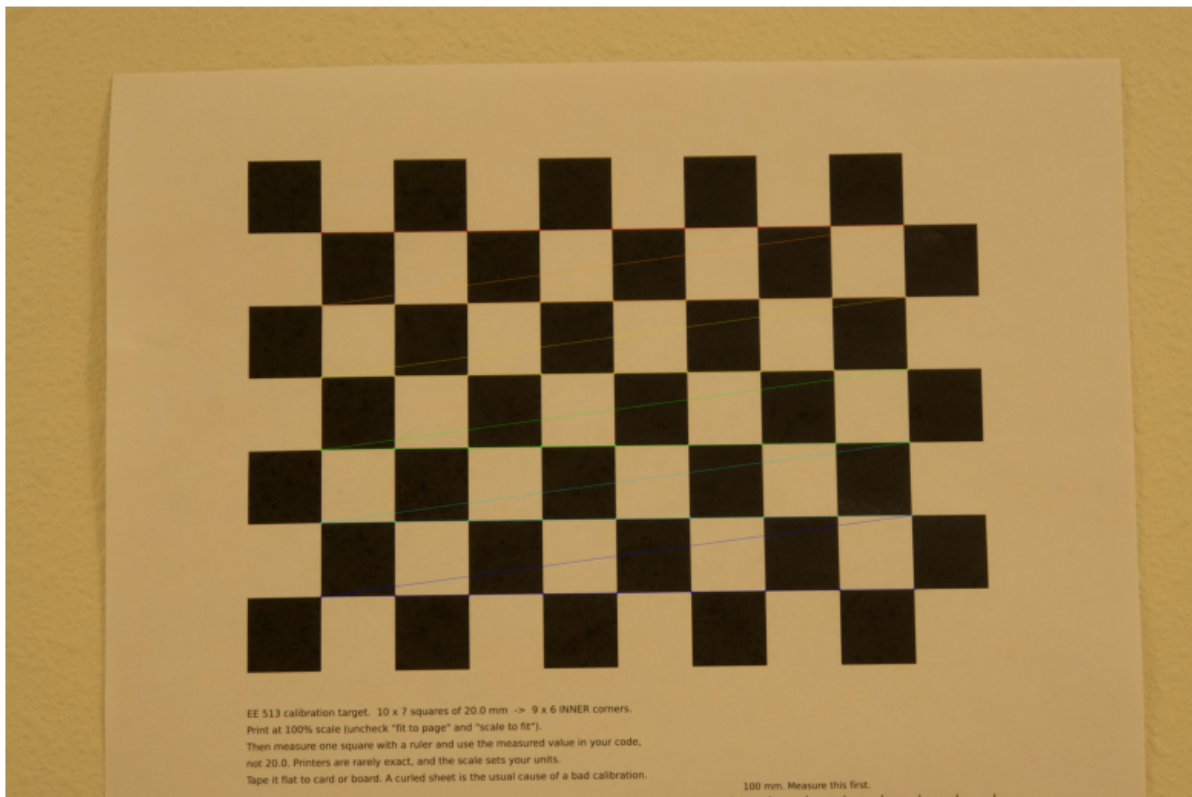
no corners: DSC_1207.jpeg

no corners: DSC_1208.jpeg

no corners: DSC_1209.jpeg

Corners found: 20 out of 24 images succeeded in finding corners.

Detected Calibration Corners



RMS Reprojection Error = 2.4053 px

K =

[1.22607047e+04 0.00000000e+00 2.76074281e+03]

[0.00000000e+00 1.22652017e+04 1.97112373e+03]

[0.00000000e+00 0.00000000e+00 1.00000000e+00]

dist = [1.47073390e-01 -4.24906676e+00 -6.96814677e-05 3.31306898e-03
6.79953116e+01]

Original Values with nominal 20.0mm measurement (prior to measuring and using 19.9mm):

24 image files found

no corners: DSC_1206.jpeg

```

no corners: DSC_1207.jpeg
no corners: DSC_1208.jpeg
no corners: DSC_1209.jpeg
Corners found: 20 out of 24 images succeeded in finding corners.
RMS Reprojection Error = 2.4053 px
K =
[[1.22607067e+04 0.00000000e+00 2.76073926e+03]
 [0.00000000e+00 1.22652035e+04 1.97112176e+03]
 [0.00000000e+00 0.00000000e+00 1.00000000e+00]]
dist = [ 1.47079413e-01 -4.24947548e+00 -6.97256988e-05
 3.31293231e-03
 6.80011427e+01]

```

These values, although very close to the 19.9mm calculated distances, differ with a very close margin, expected from a 0.01mm change.

The root mean square (RMS) reprojection error is measured in pixels, quantifying the calculated average distance between detected corners (physically, from the method) in the images and where the model projects the 3D points back onto the 2D image plane

13. What did you learn from this assignment? Deliverable: a written response.

In a RAW capture, the color filter array is applied to raw light intensity "grid". I was not aware of this, and thought that the CFA was "built in" to the camera sensor (since it's mirrorless and gets light applied to it directly). I was not expecting the Bayer mosaic to be a "single" 30x30 grid, as opposed to a 30x30x3 matrix (I have a MATLAB background and thought each RGB value would get its own 30x30 matrix, hence the three different matrices).

This became more apparent when I ran the mosaic on the Union Jack and green background image, as having picked the mostly "blue" Big Ben picture made me think that it was ONLY a light blue profile in the JPG demosaic

I also became more familiarized with OpenCV methods (mainly thanks to the notebooks provided), as I had never used it before. *14. What questions, if any, do you have for me after completing this assignment? Deliverable: a written response.* I am still a bit unsure on how a Bayer Mosaic has a CFA applied to it. How does the CFA discern between RGB values? Also, why are there 2 green channels in my camera? b'RGBG is what is reported. From the mosaic, it appears as if only one green channel is used. Why is it not a 40x40 grid?

Use of GenAI Agents: Gemini AI for "dissection" of the projection and intrinsics and calibrate your camera notebook, mainly for OpenCV method definition and applications

A "skeleton" code program structure was also generated by Gemini, and then expanded from there. Since taking ECE558, I have avoided large snippets of code in GenAI, as this approach very quickly devolves into modifying and adding unnecessary large subsections of code to what should be a simple 5-9 line routine. As such, I tried to run

every single "step" of the code to check that the method worked and was accepting the input (a single image at the beginning), before having AI expand on the next function. This can be seen on this snippet:

```
# File import
for f in files:
    img = load_bgr(f)
    if img is None:
        continue
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    if img_shape is None:
        img_shape = gray.shape[::-1]          # (width,
height)
    elif gray.shape[::-1] != img_shape:      # e.g.
portrait vs landscape
        print(f" size {gray.shape[::-1]} != {img_shape}, skipping:
{f.name}")
        continue
    # resizing array/image so that findChessboardCorners does not use
upt as many resources
    small = cv2.resize(gray, None, fx=DETECT_SCALE, fy=DETECT_SCALE,
interpolation=cv2.INTER_AREA)
    ok, corners = cv2.findChessboardCorners(small, BOARD, flags)
    if not ok:
        print(f" no corners: {f.name}")
        continue
    # As I understand it, this is similar to "scale images" where
the smaller image is mapped out
    # to larger coordinates
    corners = (corners / DETECT_SCALE).astype(np.float32) # back to
full-res pixel coords
    corners = cv2.cornerSubPix(gray, corners, (11, 11), (-1, -1),
crit)
    obj_points.append(objp)
    img_points.append(corners)
    used.append(f)
    if drawn_img is None:
        drawn_img = img.copy()
        cv2.drawChessboardCorners(drawn_img, BOARD, corners, ok)

print(f"Corners found: {len(img_points)} out of {len(files)} images
succeeded in finding corners.")
```

Where the small scaling was used to save time/resources. I benefited a lot from the generated code for this part

Claude was used to resolve tasks 10, 11, and 12 stalling, as they took upwards of 7 minutes before the Python module on my MacBook Pro completely crashed. I attempted using 15 images with the same outcome, and with the memory fix it provided, the full 24 image array was accepted and successfully analyzed through the findChessboardCorners method

